

强化学习概述（一）：RL 也是机器学习 三步骤

Actor, Environment, Reward, Return, and the Intuition of
Policy Gradient



基于李宏毅老师《机器学习 2025》课程内容整理
2026 年 6 月 9 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 45 分 01 秒

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=46>

目录

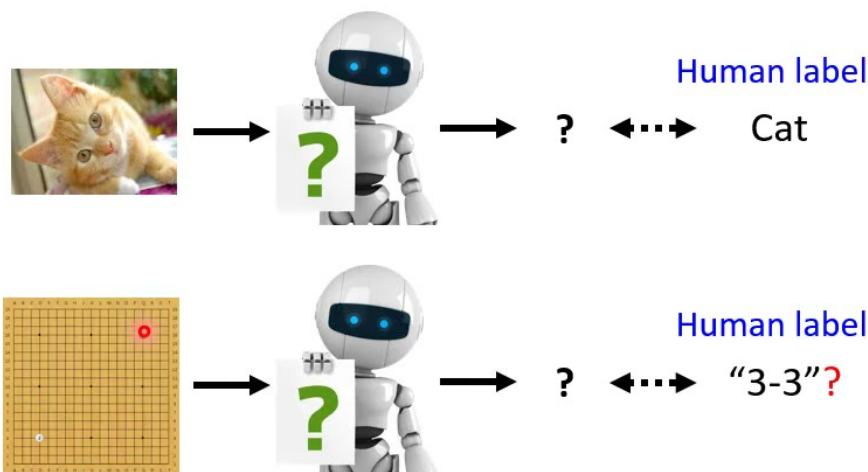
1 从监督学习走向强化学习	3
1.1 RL 中的四个基本对象	4
1.2 本章小结	5
2 例子：游戏、太空侵略者与 AlphaGo	5
2.1 本章小结	9
3 RL 也是机器学习三步骤	9
3.1 Step 1: 定义 actor, 也就是 policy network	10
3.2 Step 2: 定义目标, 最大化 return	11
3.3 Step 3: 优化 policy	12
3.4 本章小结	14
4 为什么 RL 优化困难	14
4.1 与 GAN 的类比	15
4.2 本章小结	16
5 Policy Gradient 的直觉：控制 actor	16
5.1 让 actor 更常采取某个动作	17
5.2 让 actor 避免某个动作	18
5.3 把行为资料整理成训练集	19
5.4 从二元标签到 advantage 权重	20
5.5 这一讲停在哪里	22
5.6 本章小结	23
6 总结与延伸	23
6.1 下一步要解决的问题	24

1 从监督学习走向强化学习

这节课开始进入 deep reinforcement learning。老师一开始刻意把强化学习拉回前面熟悉的框架：机器学习并不是一堆完全不同的魔法，而是常常可以被理解成“找一个函数”。在监督学习中，我们有输入、模型输出和人类标注；模型的目标是让输出接近 label。强化学习也在找函数，只是 label 不再是每一步直接给出的正确答案，而是来自模型与环境互动之后得到的 reward。

从监督学习到强化学习

Supervised Learning → RL



It is challenging to label data in some tasks.

图 1: 从监督学习到 RL: 猫狗分类可以有人类 label, 但围棋某个盘面的最佳下一手很难直接标注。¹

围棋例子说明了 RL 的动机。我们可以收集高手棋谱，把高手某一步当成 label；但那一步是否真的是该局面下最好的动作，未必可知。更一般地，在许多任务中，人类很难告诉机器“此刻正确动作是什么”，却可以让机器行动以后判断结果好坏。例如游戏赢了还是输了、得了多少分、机器人是否完成任务。这种“没有逐步标签，但有互动反馈”的场景，就是强化学习适合登场的地方。

¹视频画面时间区间：00:03:30–00:03:45。

Outline



图 2: 本节强化学习大纲: 先解释 RL 与机器学习三步骤的对应, 再进入 policy gradient、actor-critic、reward shaping 等主题。²

RL 与监督学习的关键差别

监督学习通常给定 (x, y) , 模型只要让 $f_{\theta}(x)$ 接近 y 。强化学习中, 模型先根据观察采取动作, 动作会改变环境, 环境再回传新的观察和 reward。模型没有每一步的标准答案, 只有长期互动后的好坏信号。

1.1 RL 中的四个基本对象

强化学习的基本循环包含 actor、environment、observation 和 reward。actor 是我们要学习的函数, 也常被称为 policy。environment 是 actor 互动的外部世界。environment 给 actor 一个 observation, actor 根据 observation 采取 action; action 影响 environment, environment 再给出新的 observation 和 reward。

²视频画面时间区间: 00:04:15–00:04:30。

Machine Learning ≈ Looking for a Function

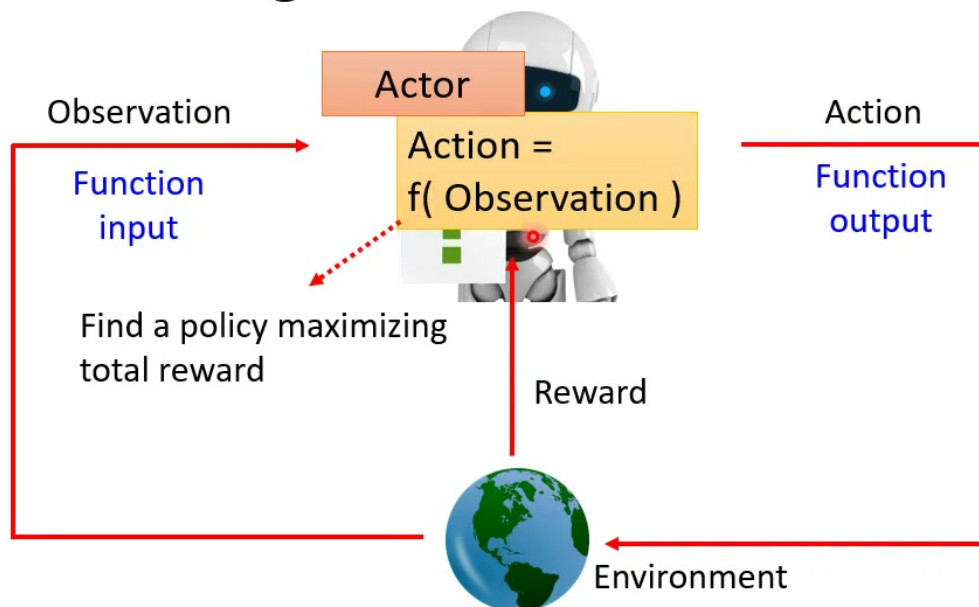


图 3: RL 的基本循环: environment 给 observation, actor 输出 action, action 影响环境, 环境再回传 reward 与新 observation。³

若用符号表示, policy 是一个从状态或观察到动作分布的函数:

$$\pi_{\theta}(a | s) = P_{\theta}(a_t = a | s_t = s).$$

其中 s_t 是第 t 步的 observation 或 state, a_t 是 actor 采取的动作, θ 是 policy network 的参数。若动作空间是离散的, $\pi_{\theta}(a | s)$ 可以理解为神经网络输出的 softmax 概率。

observation 与 state

课程中主要用 observation 解释, 因为游戏画面或棋盘局面就是 actor 实际看到的输入。严格 RL 文献中, state 往往指环境内部的完整状态, observation 指 agent 可见的信息。若环境完全可观测, 二者可以近似等同; 若不完全可观测, observation 只是 state 的一部分投影。

1.2 本章小结

强化学习仍然可以被看成“找一个函数”, 但这个函数不是把输入直接映射到固定 label, 而是在环境中持续行动, 并通过 reward 知道行动好坏。RL 的难点来自 label 的缺失、动作对未来观察的影响, 以及 reward 可能很延迟。

2 例子: 游戏、太空侵略者与 AlphaGo

为了把抽象循环具体化, 课程先用 Atari 游戏 Space Invader 作例子。actor 看到游戏画面, 动作可以是左移、右移或开火。游戏画面中的外星人、防护罩、飞船位置、分数等都组成 observation。actor 的目标不是预测某个固定 label, 而是通过一连串动作获得尽可能高的分数。

³视频画面时间区间: 00:07:00–00:07:15。

Example: Playing Video Game

- Space invader

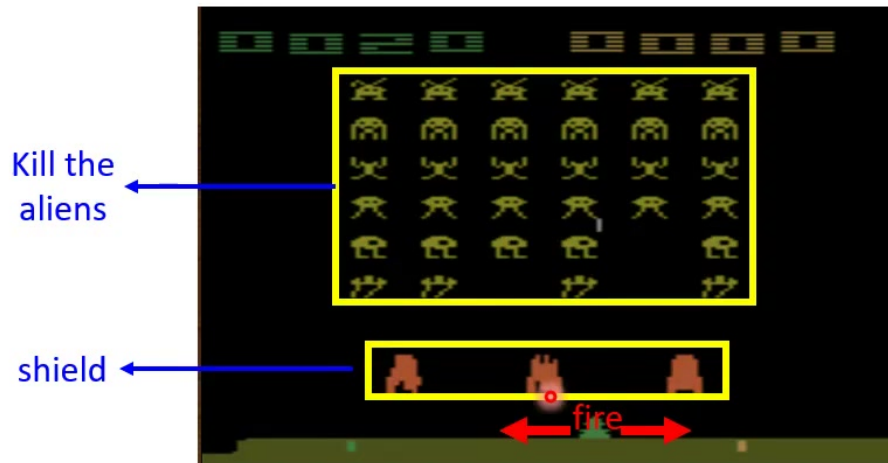


图 4: Space Invader 中, actor 的观察是游戏画面, 动作包括左移、右移与开火; 目标是杀掉外星人并保护飞船。⁴

当 actor 开火并击中外星人, environment 会给 reward, 例如 $r = 5$ 。若动作没有立刻带来分数, reward 可能是 0。游戏结束时如果飞船被摧毁, 也可能得到负面结果。重点是: actor 必须学习的不只是“当前哪一个动作看起来合理”, 而是“哪些动作序列能带来更高的总 reward”。

⁴视频画面时间区间: 00:08:00–00:08:15。

Example: Playing Video Game

Find an actor maximizing expected reward.

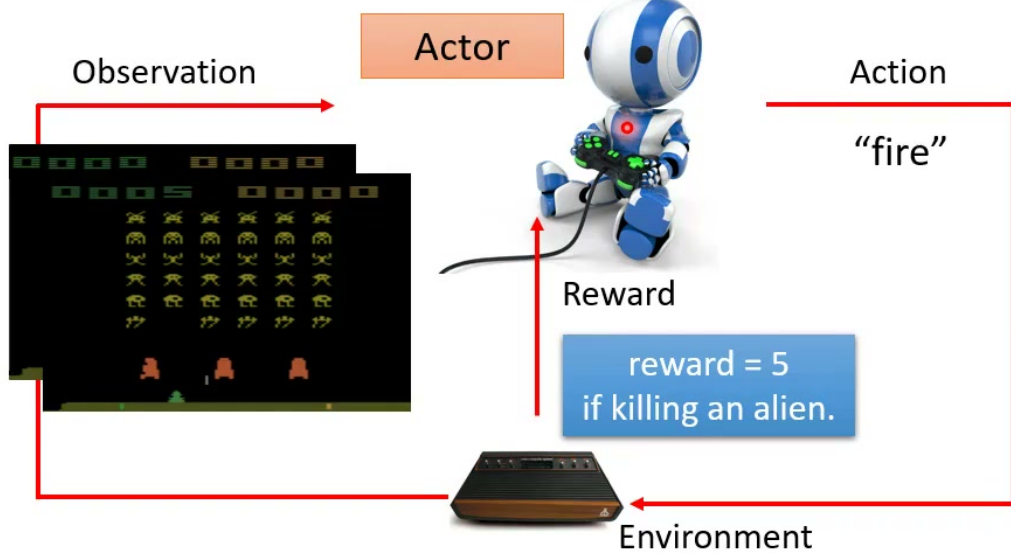


图 5: 在游戏中, actor 根据画面采取动作, 若击中外星人就得到 reward; 目标是最大化整局游戏的期望 reward。⁵

围棋的例子则强调 delayed reward。AlphaGo 的 observation 是棋盘上的黑白子位置, action 是下一步落子位置。environment 可以看成对手与围棋规则。大多数中间步骤没有即时 reward, 只有整局结束时, 赢了得到正 reward, 输了得到负 reward。

⁵视频画面时间区间: 00:11:00-00:11:15。

Example: Learning to play Go

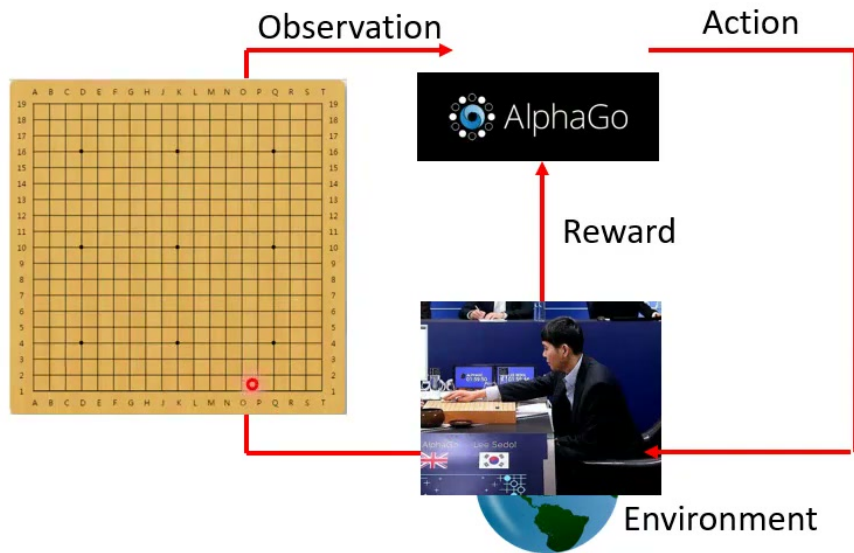


图 6: AlphaGo 设定中, observation 是棋盘局面, action 是下一步落子, environment 包含对手与规则。⁶

⁶视频画面时间区间: 00:12:15-00:12:30。

Example: Learning to play Go

Find an actor maximizing expected reward.



图 7: 围棋的 reward 很稀疏: 多数中间动作 reward 为 0, 只有游戏结束时根据胜负给出最终 reward。⁷

delayed reward 带来的 credit assignment

若最后赢棋, 究竟是哪几步促成胜利? 若最后输棋, 哪一步应该负责? 强化学习必须处理 credit assignment: 把长期结果分配回早先动作。围棋这类稀疏 reward 任务尤其困难, 因为中间几乎没有逐步指导。

2.1 本章小结

Space Invader 展示了比较直观的即时 reward: 击中外星人就加分。围棋展示了更困难的稀疏与延迟 reward: 只有整局结束时才知道胜负。两者共同说明, RL 关心的是一连串动作造成的长期结果, 而不是单一步骤的局部正确性。

3 RL 也是机器学习三步骤

老师接着把 RL 放回机器学习三步骤: 第一步, 写出带未知参数的函数; 第二步, 定义 loss 或目标函数; 第三步, 用优化方法调整参数。强化学习看起来复杂, 但仍然可以被整理成这三个步骤。

⁷ 视频画面时间区间: 00:14:00-00:14:15。

Machine Learning is so simple

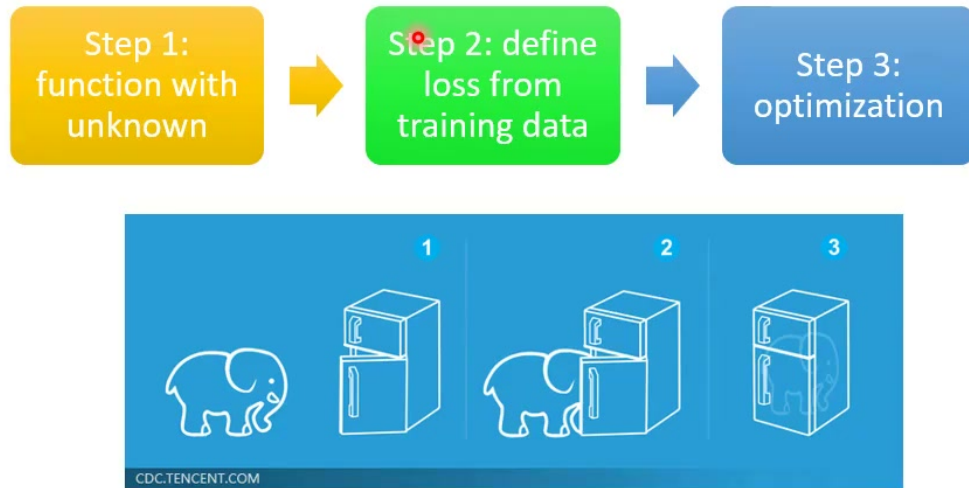


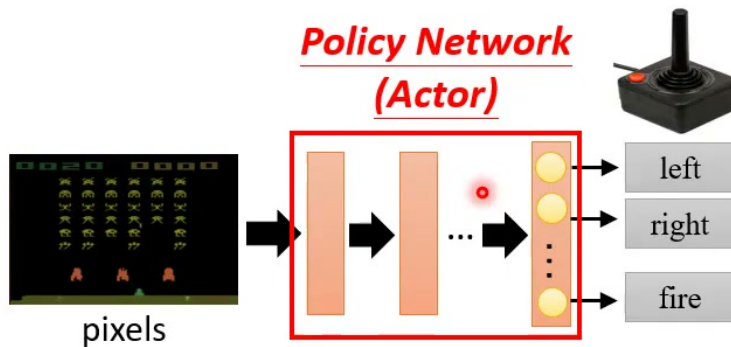
图 8: 机器学习三步骤同样适用于 RL: 定义含未知参数的函数、定义目标, 再进行优化。⁸

3.1 Step 1: 定义 actor, 也就是 policy network

第一步是定义 policy network。以游戏为例, 输入可以是画面像素矩阵, 输出层每个神经元对应一个可选动作, 例如 left、right、fire。网络输出不一定直接选择最大分数动作, 而可以形成一个动作分布, 再从中采样。

⁸视频画面时间区间: 00:14:15-00:14:30。

Step 1: Function with Unknown



- Input of neural network: the observation of machine represented as a vector or a matrix
- Output neural network : each action corresponds to a neuron in output layer

图 9: policy network 将 observation 映射到动作分数或概率; 每个输出神经元对应一个可执行 action。⁹

若输出是 logits $z_\theta(s, a)$, 可以通过 softmax 得到 policy:

$$\pi_\theta(a | s) = \frac{\exp(z_\theta(s, a))}{\sum_{a' \in \mathcal{A}} \exp(z_\theta(s, a'))}.$$

其中 $\mathcal{A}$ 是动作集合。实际执行时, 可以选择

$$a_t \sim \pi_\theta(\cdot | s_t),$$

也就是按概率采样动作, 而不总是取 $\arg \max_a \pi_\theta(a | s_t)$ 。

为什么要采样动作

采样让 policy 保持探索能力。若一开始总是选当前分数最高的动作, 模型可能永远没有机会发现其他动作其实更好。强化学习中的 exploration 与 exploitation 平衡, 正是后续算法设计的重要问题。

3.2 Step 2: 定义目标, 最大化 return

第二步是定义要最大化的量。与监督学习不同, RL 不是直接给每个 state-action pair 一个正确 label, 而是观察一整段互动。一次从开始到结束的互动称为 episode; 在 episode 中得到的 reward 总和称为 return。

⁹ 视频画面时间区间: 00:16:00-00:16:15。

Step 2: Define “Loss”

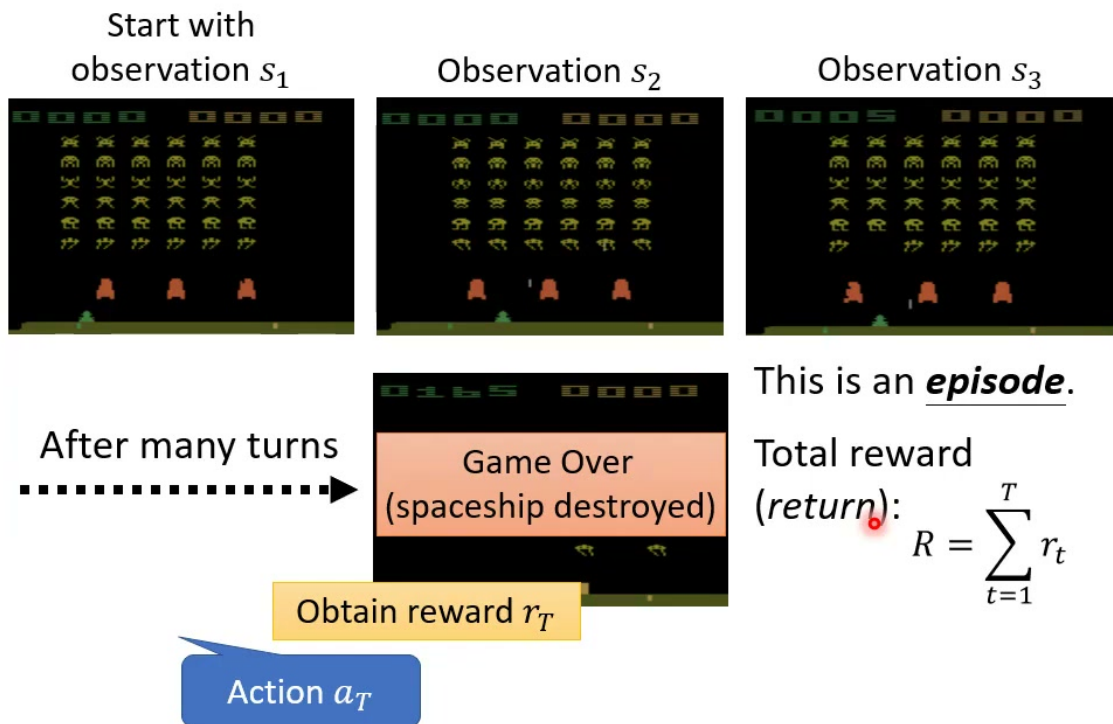


图 10: 一整局游戏形成一个 episode; 从开始到结束的 reward 总和称为 return。¹⁰

设一条 trajectory 为

$$\tau = (s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T, a_T, r_T).$$

不考虑折扣时, return 是

$$R(\tau) = \sum_{t=1}^T r_t.$$

很多 RL 文献也使用折扣回报:

$$R_\gamma(\tau) = \sum_{t=1}^T \gamma^{t-1} r_t, \quad 0 < \gamma \leq 1.$$

其中 γ 控制未来 reward 的权重。本节课的直觉讲解主要使用未折扣的总 reward。

3.3 Step 3: 优化 policy

第三步是找参数 θ , 让 actor 与环境互动时得到的 return 尽可能大。因为 actor 的动作会影响后续 state, 所以 θ 不只影响当前输出, 也影响整条 trajectory 的分布。目标可写成

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[R(\tau)], \quad \theta^* = \arg \max_{\theta} J(\theta).$$

其中 $p_\theta(\tau)$ 表示由 policy π_θ 与 environment 共同诱导出的 trajectory 分布。

¹⁰视频画面时间区间: 00:22:15–00:22:30。

Step 3: Optimization

Trajectory

$$\tau = \{s_1, a_1, s_2, a_2, \dots\}$$

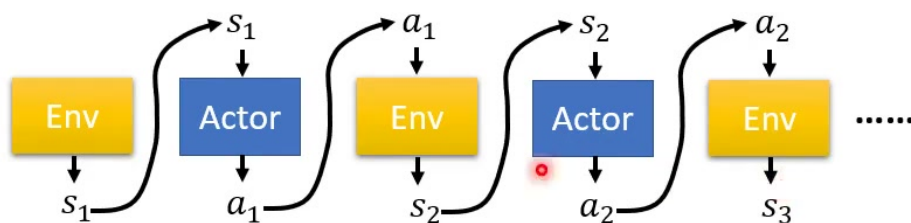


图 11: actor 与 environment 交替产生 state、action 和 reward，形成一条 trajectory。¹¹

¹¹视频画面时间区间：00:25:00-00:25:15。

Step 3: Optimization

Trajectory

$$\tau = \{s_1, a_1, s_2, a_2, \dots\}$$

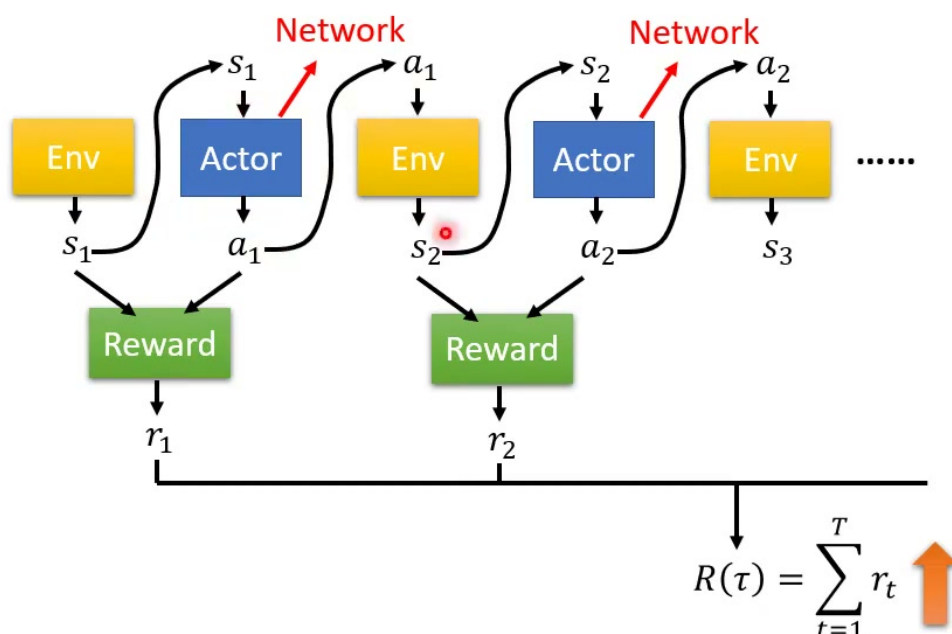


图 12: RL 优化目标：调整 policy network 的参数，使 trajectory 的 return 越大越好。¹²

3.4 本章小结

RL 的三步骤与一般机器学习一致：定义 policy network、定义 return 目标、优化参数。差别在于目标函数不是静态数据集上的 loss，而是 policy 与环境互动后得到的期望 return；policy 的参数会改变未来数据分布，这让优化更难。

4 为什么 RL 优化困难

如果 environment 和 reward 都是可微、确定且已知的神经网络，那么我们似乎可以直接对 $J(\theta)$ 做反向传播。但现实中的 RL 不满足这个理想条件。environment 常常是黑盒，例如游戏机、真实机器人、围棋对手或在线用户；reward 函数也未必可微；更麻烦的是，互动过程本身有随机性。

¹² 视频画面时间区间：00:27:15–00:27:30。

Step 3: Optimization

Trajectory

$$\tau = \{s_1, a_1, s_2, a_2, \dots\}$$

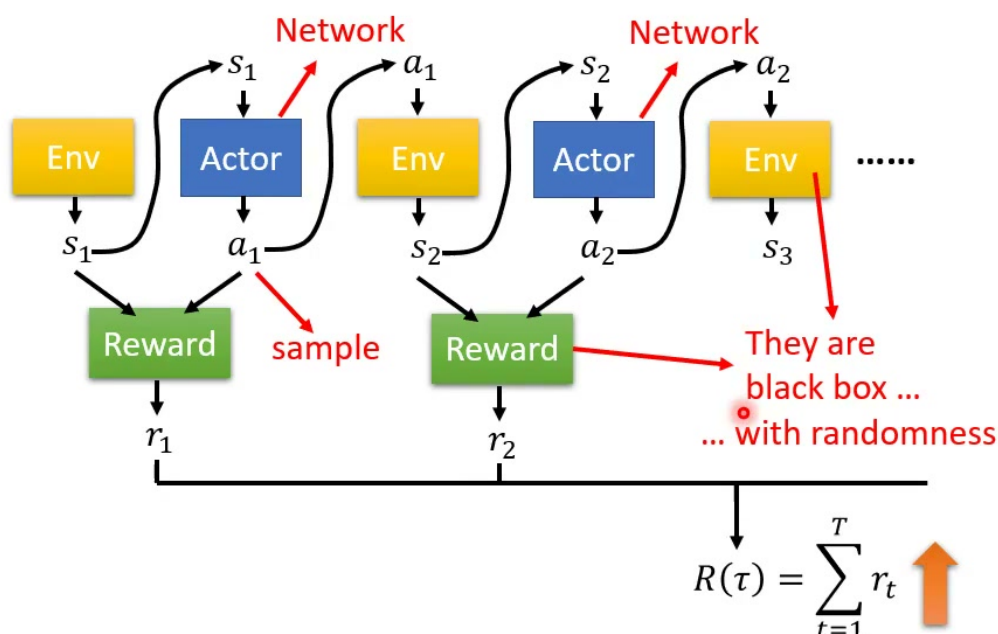


图 13: environment 和 reward 通常是黑盒，且 trajectory 采样带有随机性；同样的 policy 不一定每次产生同样结果。¹³

这种黑盒随机性意味着，不能简单地把整个环境接到神经网络后面做端到端微分。我们只能让 actor 去试、收集 trajectory、观察 reward，再根据这些经验调整 policy。这也解释了为什么 RL 的训练通常不稳定、样本效率低，而且实验结果方差很大。

4.1 与 GAN 的类比

课程用 GAN 作类比：在 GAN 里，generator 产生样本，discriminator 给出评价；训练 generator 时，我们希望 discriminator 的输出更高。RL 中 actor 有点像 generator，environment 和 reward 有点像 discriminator；我们调 actor，让 reward 更高。

¹³ 视频画面时间区间：00:29:45–00:30:00。

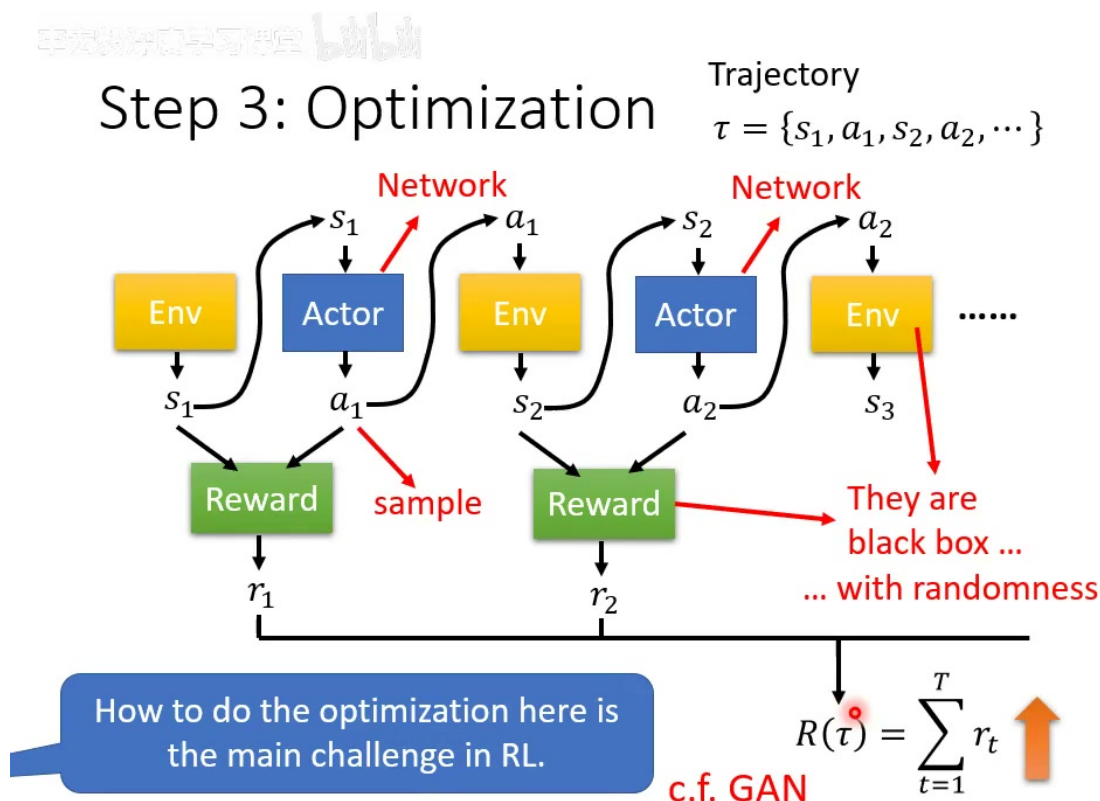


图 14: RL 优化可类比 GAN: actor 像 generator, environment 与 reward 像评价系统; 但 RL 的评价系统通常是不可微黑盒。¹⁴

关键差别在于, GAN 的 discriminator 通常也是神经网络, 我们知道它内部结构, 能把梯度传回 generator; RL 的 environment 不一定是神经网络, 也不一定可微。即便在电子游戏中, 环境内部也可能有随机数; 在围棋中, 对手如何回应更不受我们控制。因此 policy optimization 需要专门方法, policy gradient 就是其中最基本的一类。

RL 代码容易跑, 稳定结果不容易

老师提到 RL 常让人感觉又难又容易: 网上有很多可运行代码, 但同样 network 每次测试结果可能都不同。随机性和黑盒环境使 RL 实验方差很大, 因此评估时通常需要多次 seed、平均回报和置信区间。

4.2 本章小结

RL 优化困难的根源是: 目标依赖整条 trajectory, trajectory 又由 policy 与黑盒环境共同产生; 环境不可微、reward 延迟且采样有随机性。policy gradient 的目的, 就是在这种不能直接反向传播的情况下, 仍然找到调整 actor 的方向。

5 Policy Gradient 的直觉: 控制 actor

课程最后进入 policy gradient, 但没有从完整推导开始, 而是用一个更贴近监督学习的角度讲: 如果我们知道在某个 observation s 下希望 actor 采取动作 $\hat{a}$, 那就可以像训练分类器一样, 用 cross-entropy 让 actor 更倾向于输出 $\hat{a}$ 。

¹⁴ 视频画面时间区间: 00:31:00-00:31:15。

How to control your actor

- Make it take (or don't take) a specific action $\hat{a}$ given specific observation s .

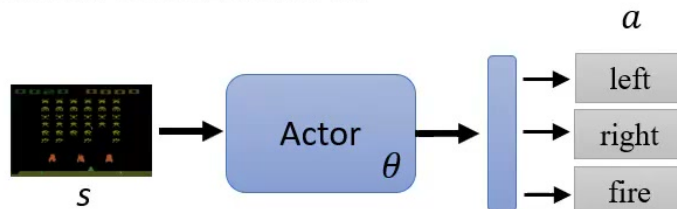


图 15: 进入 policy gradient: 本节用“如何控制 actor”的角度解释 policy optimization。¹⁵

5.1 让 actor 更常采取某个动作

若希望 actor 在 state s 下采取动作 $\hat{a}$, 可以把 $\hat{a}$ 当成 label, 计算 cross-entropy:

$$e(s, \hat{a}; \theta) = -\log \pi_{\theta}(\hat{a} | s).$$

最小化 e 会增大 $\pi_{\theta}(\hat{a} | s)$, 也就是让 actor 更常采取 $\hat{a}$ 。

¹⁵ 视频画面时间区间: 00:35:45-00:36:00。

How to control your actor

- Make it take (or don't take) a specific action $\hat{a}$ given specific observation s .

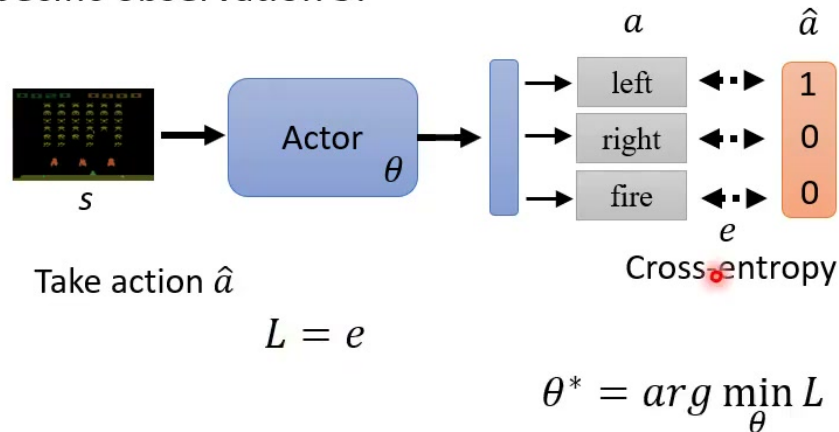


图 16: 若希望 actor 在 observation s 下采取动作 $\hat{a}$, 可把动作当成 label, 用 cross-entropy 训练。¹⁶

这和训练图像分类器很像：输入是图片，label 是类别；在 RL 中，输入是 observation，label 是希望采取的动作。区别是，RL 中这个 label 不是人类预先标好的，而是需要从互动经验中判断出来。

5.2 让 actor 避免某个动作

如果希望 actor 在某个 observation s' 下不要采取动作 $\hat{a}'$, 可以用相反方向的目标。直觉上，最小化

$$-e(s', \hat{a}'; \theta) = \log \pi_{\theta}(\hat{a}' | s')$$

会压低 $\pi_{\theta}(\hat{a}' | s')$, 让 actor 不倾向于该动作。把“希望做”和“不希望做”放在一起，可以写成

$$L(\theta) = e(s, \hat{a}; \theta) - e(s', \hat{a}'; \theta).$$

¹⁶ 视频画面时间区间：00:37:00–00:37:15。

How to control your actor

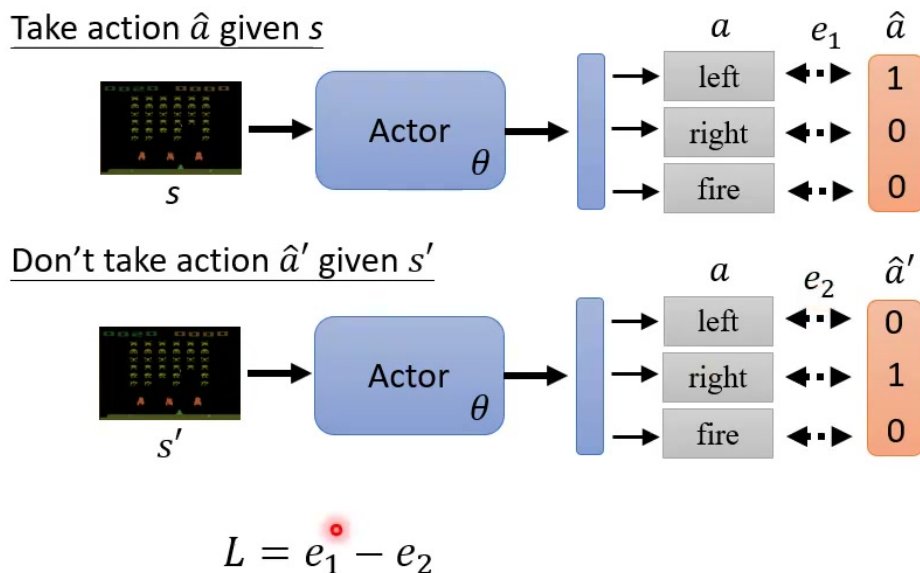


图 17: 同时控制正向与负向行为: 对希望采取的动作降低 cross-entropy, 对不希望采取的动作提高 cross-entropy。¹⁷

这是 policy gradient 的直觉版

直接最小化负 cross-entropy 只是帮助理解“鼓励或抑制某个动作”的方向。实际 policy gradient 会从期望 return 的梯度推导出加权 log-probability 目标, 并会加入 baseline、advantage、entropy regularization 等稳定训练技巧。

5.3 把行为资料整理成训练集

如果我们收集到许多 (s_n, a_n) , 并知道每个动作是“希望执行”还是“不希望执行”, 就可以像分类任务一样训练 actor。课程用 +1 表示希望做, 用 -1 表示不希望做:

$$\mathcal{D} = \{(s_n, a_n, y_n)\}_{n=1}^N, \quad y_n \in \{+1, -1\}.$$

对应的直觉 loss 可以写成

$$L(\theta) = \sum_{n=1}^N y_n e(s_n, a_n; \theta).$$

若 $y_n = +1$, 最小化会鼓励动作; 若 $y_n = -1$, 最小化会抑制动作。

¹⁷ 视频画面时间区间: 00:39:00-00:39:15。

How to control your actor

Training Data

$\{s_1, \hat{a}_1\}$	+1	Yes
$\{s_2, \hat{a}_2\}$	-1	No
$\{s_3, \hat{a}_3\}$	+1	Yes
$\vdots$	$\vdots$	
$\{s_N, \hat{a}_N\}$	-1	No



$$L = +e_1 - e_2 + e_3 \cdots - e_N$$

$$\theta^* = \arg \min_{\theta} L$$

图 18: 把 actor 控制问题整理成训练资料: 某些 state-action pair 标成 Yes, 某些标成 No。¹⁸

5.4 从二元标签到 advantage 权重

实际情况不只是“好”或“不好”。某个动作可能非常好、稍微好、稍微坏或非常坏。因此课程把 +1/-1 推广成一个实数权重 A_n 。 A_n 越大, 越希望 actor 在 s_n 下采取 a_n ; A_n 越小甚至为负, 越希望 actor 避免该动作。

¹⁸ 视频画面时间区间: 00:41:45-00:42:00。

How to control your actor

Training Data

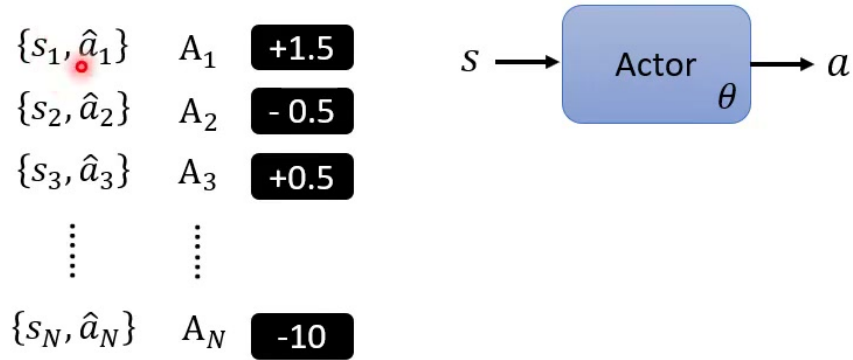


图 19: 每个 state-action pair 不只有二元好坏, 而可以有强弱不同的权重 A_n 。¹⁹

于是 actor 的训练目标可以写成

$$L(\theta) = \sum_{n=1}^N A_n e(s_n, a_n; \theta) = - \sum_{n=1}^N A_n \log \pi_{\theta}(a_n | s_n).$$

最小化这个 loss 等价于: 对 $A_n > 0$ 的动作, 提高其概率; 对 $A_n < 0$ 的动作, 降低其概率。这个形式已经很接近 policy gradient 中常见的 advantage-weighted log-probability objective。

¹⁹视频画面时间区间: 00:42:30-00:42:45。

How to control your actor

Training Data

$\{s_1, \hat{a}_1\}$	A_1	+1.5
$\{s_2, \hat{a}_2\}$	A_2	-0.5
$\{s_3, \hat{a}_3\}$	A_3	+0.5
$\vdots$	$\vdots$	
$\{s_N, \hat{a}_N\}$	A_N	-10



$$L = \sum A_n e_n$$

$$\theta^* = \arg \min_{\theta} L$$

图 20: 用 A_n 加权每个 cross-entropy 项: 正权重鼓励动作, 负权重抑制动作。²⁰

policy gradient 的一行直觉

让 actor 回顾自己做过的动作: 如果某个动作后来带来好结果, 就增加它在相似状态下被采样的概率; 如果带来坏结果, 就降低它的概率。数学上, 这会变成用回报或 advantage 加权的 $\log \pi_{\theta}(a | s)$ 。

5.5 这一讲停在哪里

课程最后留下两个问题。第一, A_n 要怎么产生? 也就是如何判断某个 state-action pair 到底该被鼓励还是抑制, 以及鼓励程度是多少。第二, 训练资料 (s_n, a_n) 从哪里来? 它们显然不能像监督学习那样由人工完整标注, 而要来自 actor 与环境的互动轨迹。

²⁰ 视频画面时间区间: 00:44:00-00:44:15。

How to control your actor

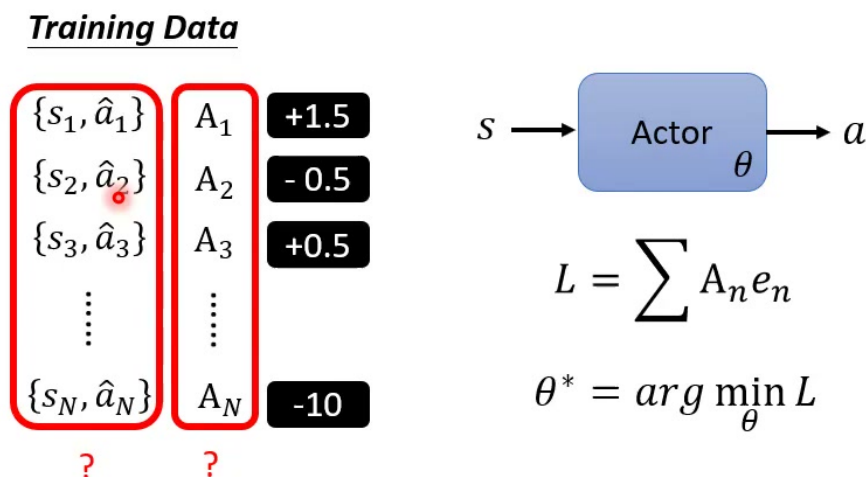


图 21: 本节结尾的问题: 训练 actor 所需的 (s_n, a_n) 与权重 A_n 从哪里来? 这正是后续 policy gradient 要解决的核心。²¹

5.6 本章小结

policy gradient 可以先用监督学习直觉理解: 如果知道某个动作该做, 就用 cross-entropy 提高它的概率; 如果知道不该做, 就降低它的概率; 如果知道好坏程度, 就用权重 A_n 调整力度。真正的 RL 问题是, A_n 和训练样本都必须从互动经验中估计, 而不是由静态标签直接给出。

6 总结与延伸

本节课完成了强化学习入门的第一层框架。RL 仍然是机器学习三步骤: 定义 actor, 定义目标, 优化参数。actor 是 policy network, 输入 observation, 输出 action 分布; environment 根据 action 变化, 并回传 reward。一次完整互动形成 trajectory 或 episode, reward 总和称为 return。优化目标是最大化 policy 诱导出的期望 return:

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[R(\tau)].$$

RL 与监督学习最大的不同, 在于没有每一步的正确答案。label 隐藏在长期互动结果里, 而且 action 会改变之后看到的数据。Space Invader 中 reward 可以相对频繁地出现; 围棋中 reward 可能只在终局出现。这样的 delayed reward 让 credit assignment 变得困难, 也让 policy optimization 不能简单套用普通 backpropagation。

从 policy gradient 的角度看, 训练 actor 可以被理解为加权分类: 对某个 state-action pair, 如

²¹ 视频画面时间区间: 00:44:45-00:45:00。

果权重 A_n 为正，就提高该动作概率；如果 A_n 为负，就降低该动作概率。于是

$$L(\theta) = - \sum_{n=1}^N A_n \log \pi_{\theta}(a_n | s_n)$$

成为一个很有解释力的形式。它把 RL 的“长期好坏”转换成对局部动作概率的鼓励或惩罚。

6.1 下一步要解决的问题

这节课刻意停在最关键的缺口上：我们还不知道 A_n 怎么来，也不知道哪些 (s_n, a_n) 应该进入训练集。后续 policy gradient 会说明，trajectory 是由当前 policy 采样出来的，而 A_n 可以由 return、baseline 或 advantage function 估计。这会把“鼓励好动作、抑制坏动作”的直觉，变成可以实际训练的算法。

这一讲的最重要收获

不要把强化学习想成完全脱离前面课程的新世界。它仍然是在训练函数，只是训练信号来自与环境互动后的 reward。理解 actor、trajectory、return 和加权 cross-entropy，就抓住了 policy gradient 的入口。